Prométhée

Guide d'installation et de démarrage

Pile technique Docker complète — v3.0

FastAPI · React · Qdrant · Garage S3 · Docker Compose

version 1.0 – avril 2026

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0

Sommaire

Sommaire....2
1. Vue d'ensemble....3
1.1 Architecture de la pile technique....3
1.2 Ordre de démarrage....3
2. Prérequis....4
2.1 Logiciels requis....4
2.2 Ports réseau à libérer....4
3. Installation pas à pas....5
3.3 Récupération du GARAGE_NODE_ID (étape critique)....6
1.a — Démarrer uniquement les services Garage....7
1.b — Interroger Garage pour obtenir le Node ID....7
1.c — Lire la sortie....7
1.d — Renseigner GARAGE_NODE_ID dans .env....7
4. Référence des variables d'environnement....9
4.1 Application....9
4.2 Garage (stockage S3)....9
4.3 Qdrant (base vectorielle)....9
5. Dépannage....10
5.1 garage-init s'arrête avec une erreur....10
5.2 Garage n'est jamais healthy....10
5.3 Le Node ID change après un redémarrage....10
5.4 L'application n'est pas accessible sur le port 8000....10
5.5 Réinitialiser complètement l'installation....10
6. Référence rapide des commandes....11

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0

1. Vue d'ensemble

Prométhée AI est un assistant IA web complet composé de cinq services Docker orchestrés par Docker Compose. Ce guide décrit l'installation et le premier démarrage, en particulier la partie la plus délicate : la récupération et la configuration de l'identifiant du nœud Garage (GARAGE_NODE_ID).

1.1 Architecture de la pile technique

La pile technique se compose des services suivants :

Service	Rôle et port(s) exposé(s)
promethee	Application principale FastAPI + React (port 8000)
qdrant	Base vectorielle pour le RAG et la mémoire long terme (ports 6333 / 6334)
garage	Stockage objets compatible S3 — VFS utilisateur (port 3900 / 3901)
garage-config	Service one-shot Alpine : génère garage.toml depuis le template
garage-init	Service one-shot : configure le nœud Garage et crée le bucket S3

1.2 Ordre de démarrage

Docker Compose garantit l'ordre suivant via les dépendances déclarées :

- garage-config démarre en premier et génère `garage.toml` dans un volume partagé.
- garage démarre une fois `garage-config` terminé avec succès, puis passe en mode `healthy`.
- garage-init attend que `garage` soit sain avant de configurer le nœud et de créer le bucket.
- qdrant et `promethee` démarrent en parallèle une fois Garage opérationnel.

⚠️ **Attention :** Le GARAGE_NODE_ID doit impérativement être renseigné dans le fichier .env avant le premier lancement complet. Sans lui, le service garage-init s'arrête en erreur et l'application ne peut pas démarrer.

2. Prérequis

2.1 Logiciels requis

Logiciel	Version minimale recommandée
Docker Engine	24.x ou supérieur
Docker Compose	v2.x (plugin intégré à Docker Desktop)
Git	Toute version récente
openssl	Pour générer les secrets (inclus sur Linux/macOS)

2.2 Ports réseau à libérer

Vérifiez qu'aucun autre service n'occupe les ports suivants sur l'hôte :

Port	Service
8000	Application Prométhée
6333 / 6334	Qdrant (API REST + gRPC)
3900 / 3901	Garage (API S3 + RPC interne)

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0

3. Installation pas à pas

Suivez les cinq étapes dans l'ordre. **Ne passez pas à l'étape suivante si une étape échoue.**

1 Cloner le dépôt

Récupérez le code source depuis le dépôt Git :

```
git clone https://github.com/Ktulu-Analog/promethee.git
cd promethee
```

2 Créer et éditer le fichier .env

Copiez le fichier d'exemple fourni :

```
cp .env.example .env
```

Ouvrez .env dans votre éditeur et renseignez au minimum les variables obligatoires :

Variable	Description / valeur à fournir
PROMETHEE_SECRET_KEY	Clé secrète applicative — remplacer la valeur par défaut
GARAGE_RPC_SECRET	Secret partagé Garage — générer avec : openssl rand -hex 32
GARAGE_ACCESS_KEY	Identifiant clé S3 (format : GK + 24 hex, ex : GKdf05b4d85d5dbb3f1cce49)
GARAGE_SECRET_KEY	Secret clé S3 (64 hex) — générer avec : openssl rand -hex 32
GARAGE_NODE_ID	Laissez vide pour l'instant — voir Étape 3 ci-dessous
GARAGE_BUCKET	Nom du bucket (défaut : promethee-vfs)
QDRANT_API_KEY	Laissez vide pour désactiver l'authentification Qdrant

ℹ️ **Info : Commande openssl à exécuter pour générer les secrets : openssl rand -hex 32**

3.3 Récupération du GARAGE_NODE_ID (étape critique)

C'est l'étape la plus délicate. Le GARAGE_NODE_ID est un identifiant hexadécimal de 64 caractères que Garage génère lui-même au premier démarrage. Il est stocké dans les métadonnées persistantes du nœud et doit être connu pour configurer le cluster. Voici la procédure complète :

3	Démarrer Garage seul et récupérer le Node ID

1.a — Démarrer uniquement les services Garage

Lancez d'abord uniquement la chaîne Garage (sans l'application Prométhée ni Qdrant) :

```
docker compose up -d garage
```

Cette commande démarre automatiquement garage-config (qui génère garage.toml) puis garage.

Attendez que le service Garage soit en état healthy. Vous pouvez vérifier avec :

```
docker compose ps
```

✅ **Résultat attendu :** La colonne STATUS affiche : promethee-garage Up X seconds (healthy)

1.b — Interroger Garage pour obtenir le Node ID

Exécutez la commande suivante dans le conteneur Garage en cours d'exécution. Le binaire /garage fonctionne directement, sans shell intermédiaire (image distroless) :

```
docker exec promethee-garage /garage -c /etc/garage/garage.toml node id
```

1.c — Lire la sortie

La commande affiche une sortie similaire à celle-ci :

```
====================  Garage node ID  ====================

56f7d01ee626b9672eb88de512607f13a79c53cb90b8f5d2195bd40cf6b147d6@garage:390
1

==========================================================
```

📋 **À copier : Copiez uniquement la partie hexadécimale, AVANT le symbole @ (arobase). Ne copiez pas l'adresse @garage:3901 qui suit.**

Dans l'exemple ci-dessus, la valeur à copier est :

```
56f7d01ee626b9672eb88de512607f13a79c53cb90b8f5d2195bd40cf6b147d6
```

1.d — Renseigner GARAGE_NODE_ID dans .env

Ouvrez votre fichier .env et renseignez la ligne GARAGE_NODE_ID :

```
GARAGE_NODE_ID=56f7d01ee626b9672eb88de512607f13a79c53cb90b8f5d2195bd40cf6b1
47d6
```

⚠️ **Attention : Le Node ID est propre à chaque installation. Il change si vous supprimez le volume garage_meta. Dans ce cas, répétez l'Étape 3 complète.**

4 Lancer la pile technique complète

Une fois GARAGE_NODE_ID renseigné dans .env, lancez l'intégralité de la pile technique :

```
docker compose up -d --build
```

Docker Compose va :

- construire l'image Prométhée (FastAPI + React) et l'image garage-init
- relancer garage-config si nécessaire pour régénérer garage.toml
- attendre que Garage soit healthy puis lancer garage-init qui :
  - assigne la layout du nœud (zone dc1, capacité 1 Go)
  - importe la clé d'accès S3 dans Garage
  - crée le bucket et lui attribue les permissions read/write/owner
- démarrer Qdrant et l'application Prométhée une fois tout cela prêt

Le premier build peut prendre plusieurs minutes (compilation du frontend React, installation des paquets Python, téléchargement de KaTeX et Mermaid).

ℹ️ **Info : Suivez les logs en temps réel avec : docker compose logs -f**

5 Vérifier le bon fonctionnement

Contrôlez l'état de tous les services :

```
docker compose ps
```

✅ **Résultat attendu : Tous les services permanents (promethee, qdrant, garage) doivent afficher le statut Up ... (healthy). Les services one-shot (garage-config, garage-init) affichent Exited (0).**

Accédez ensuite à l'application dans votre navigateur :

Interface	URL d'accès
Application Prométhée	http://localhost:8000
API Garage (S3)	http://localhost:3900
Tableau de bord Qdrant	http://localhost:6333/dashboard

4. Référence des variables d'environnement

Tableau complet des variables disponibles dans .env. Les variables marquées « **Obligatoire »** doivent absolument être définies pour que l'application fonctionne.

4.1 Application

Variable	Valeur par défaut / Description
PROMETHEE_SECRET_KEY	Obligatoire — clé secrète JWT/session. Générer avec : openssl rand -hex 32
SERVER_PORT	8000 — port exposé sur l'hôte
ALLOWED_ORIGINS	["http://localhost:8000"] — origines CORS autorisées (format JSON)

4.2 Garage (stockage S3)

Variable	Valeur par défaut / Description
GARAGE_RPC_SECRET	Obligatoire — secret RPC partagé entre nœuds. Générer avec openssl rand -hex 32
GARAGE_NODE_ID	Obligatoire — ID hexadécimal 64 chars du nœud (voir Étape 3)
GARAGE_ACCESS_KEY	garagekeyid — format GK + 24 hex obligatoire
GARAGE_SECRET_KEY	garagesecretkey — 64 hex recommandé. Générer avec openssl rand -hex 32
GARAGE_BUCKET	promethee-vfs — nom du bucket S3 créé automatiquement

4.3 Qdrant (base vectorielle)

Variable	Valeur par défaut / Description
QDRANT_API_KEY	Vide = authentification désactivée. Définir une valeur pour la sécuriser.

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0

5. Dépannage

5.1 garage-init s'arrête avec une erreur

Le message d'erreur le plus fréquent est :

```
ERREUR : GARAGE_NODE_ID n'est pas défini dans votre .env.
Suivez l'étape 1 du README :
  docker exec promethee-garage /garage -c /etc/garage/garage.toml node id
Puis renseignez GARAGE_NODE_ID dans votre .env et relancez.
```

Cause : la variable GARAGE_NODE_ID est absente ou vide dans .env.

Solution : reprendre l'Étape 3 de ce guide, récupérer le Node ID et l'ajouter dans .env, puis relancer avec docker compose up -d.

5.2 Garage n'est jamais healthy

Vérifiez les logs du service :

```
docker compose logs garage
```

- Si le log indique une erreur de lecture de `garage.toml` : vérifiez que le service `garage-config` s'est bien terminé (`Exited (0)`).
- Si le log indique un conflit de port : vérifiez qu'aucune autre instance Garage ou application n'utilise les ports 3900 ou 3901.

5.3 Le Node ID change après un redémarrage

Le Node ID est stable tant que le volume Docker garage_meta existe. Il change uniquement si vous supprimez ce volume (par exemple avec docker compose down -v ou docker volume rm).

> ⚠️ **Attention : Ne jamais utiliser docker compose down -v en production : cela supprime toutes les données persistantes (métadonnées Garage, données Qdrant, données applicatives).**

5.4 L'application n'est pas accessible sur le port 8000

- Vérifiez que le service promethee est bien `Up (healthy)` avec `docker compose ps`.
- Consultez les logs : `docker compose logs promethee`.
- Vérifiez que garage-init s'est terminé avec succès (`Exited (0)`) — promethee dépend de garage qui lui-même dépend de garage-init.

5.5 Réinitialiser complètement l'installation

> ⚠️ **Attention : Cette procédure efface TOUTES les données. À n'utiliser qu'en développement.**

```
docker compose down -v   # supprime les conteneurs ET les volumes
# Puis reprendre depuis l'Étape 3 (nouveau Node ID généré)
```

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0

6. Référence rapide des commandes

Commande	Description
docker compose up -d garage	Démarrer uniquement Garage (pour récupérer le Node ID)
docker exec promethee-garage /garage -c /etc/garage/garage.toml node id	Afficher le Node ID du nœud Garage
docker compose up -d --build	Démarrer (ou redémarrer avec rebuild) la stack complète
docker compose ps	État de tous les services
docker compose logs -f	Logs en temps réel de tous les services
docker compose logs -f promethee	Logs en temps réel de l'application
docker compose logs -f garage-init	Logs de l'initialisation Garage (diagnostic)
docker compose stop	Arrêter tous les services (volumes conservés)
docker compose down	Arrêter et supprimer les conteneurs (volumes conservés)
docker compose down -v	Arrêter, supprimer conteneurs ET volumes (⚠️ perte de données)
openssl rand -hex 32	Générer un secret aléatoire sécurisé (GARAGE_RPC_SECRET, etc.)

Pierre COUGET 2026 – logiciel sous licence AGPL 3.0